

Embedded Computer Vision Exercise 3 Report

Phil Orlando
Graduate Student, Department of Electrical Engineering
University of Colorado Boulder

All source code used for this assignment is available at <https://github.com/Pmorlando/ECV-Orlando/tree/main/Ex3> and if issues downloading or viewing are encountered, please contact me. I am not the most experienced with Github but happy to learn of mistakes I am making.

1. The key points of the paper are showing the power of picture and video processing in the form of interactive media like Augmented Reality (AR) and Virtual Reality (VR). The paper explores the fundamentals of interactive media, which is when audio, video, and rendered graphics are often integrated to form content for consumers to see, hear, and even interact with. Then the paper explains a little into the process for making interactive media including integration of cloud and mobile systems, video encoding, graphical rendering, and user interactions to all come together and create a product for users to enjoy. The paper then walks through some practice to get started with the digital encoding by explaining the macro blocks and Discrete Fourier Transform we discussed in class. Then the paper shows what now is AI image generation in the form of 3D rendering animations. Finally, the paper explains AR uses like the 1st down line on football games broadcast on tv and also AR goggles examples including vehicle repair where the AR goggles can find and display part information for the user.

The possibility of generated images and video I think is even more of a concern today with how well AI can generate images and videos. I forgot what the video was but a viral video from a few months ago showing some CCTV footage showed some unbelievable event which my skeptical mind immediately said was AI generated to later find out I was wrong. I think with further advancement in AI, higher quality video can become indistinguishable from real videos. So yes, that was a valid concern then and is becoming a more valid concern by the general public.

2. Using the Hough lines, I think it will be useful for my final project idea which I am planning to be a blackjack assistant that can read cards and once I know what the cards are offer recommendations for hitting or standing. This could be used to identify the card edges like in figure 1 where the outside of the card is mostly recognized by the Hough lines. This could be used to narrow the search on the Canny edge detection for how many cards are on the table at a time and to narrow the area of the image to search for the card value.

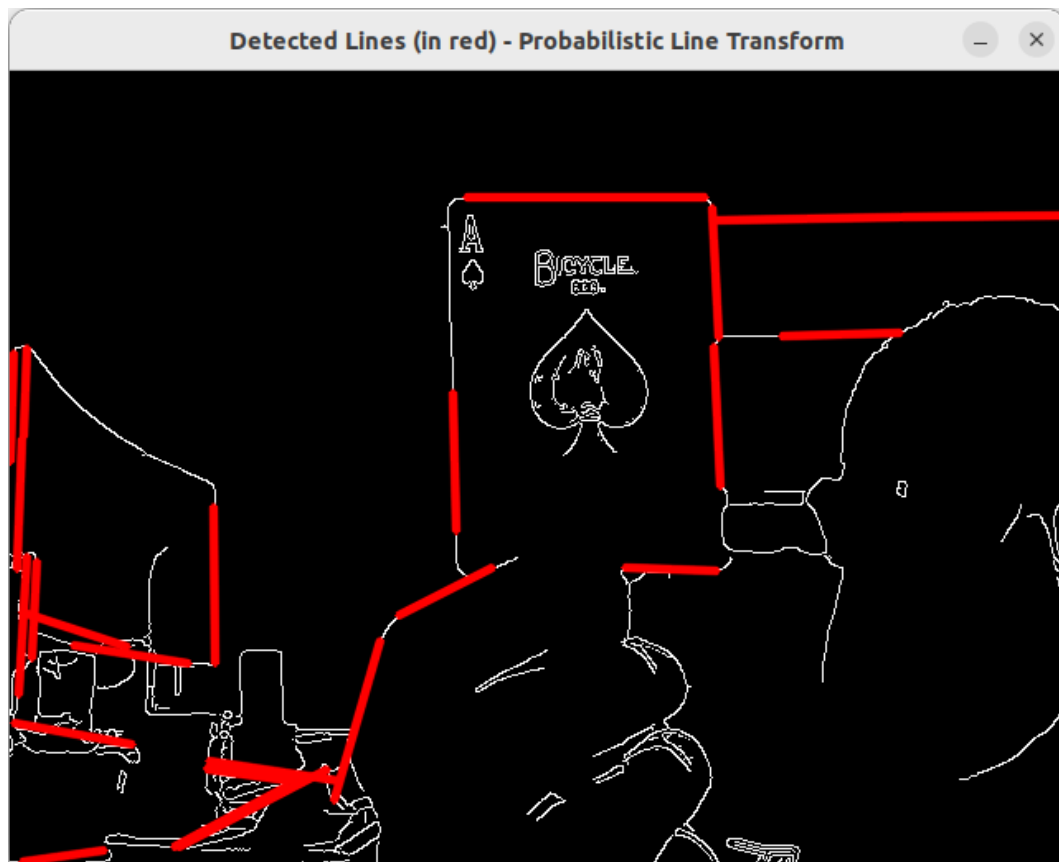


Figure (1) Hough lines and Canny edge detection on playing card.

3. I removed the canny edge detection for the Hough circles and specifically used playing cards with the numbers that are circular. This could help in the future in my final project to detect what the value of the card is. It isn't perfect because it is searching for near perfect round circles, but it is still detected portions of the 6, 8 and 9 pretty well as seen in figure 2.



Figure (2) Hough Circle on playing cards.

4. To remove the background of the laser with clutter the difference in frames method works pretty well removing all the nonmoving books for a majority of the frames but picked up on the reflections of the moving laser point. Overall, it can recognize the movement but for the intermittent frames of no movement it fails to track. This is a good start but needs more complex methods for more robust tracking.

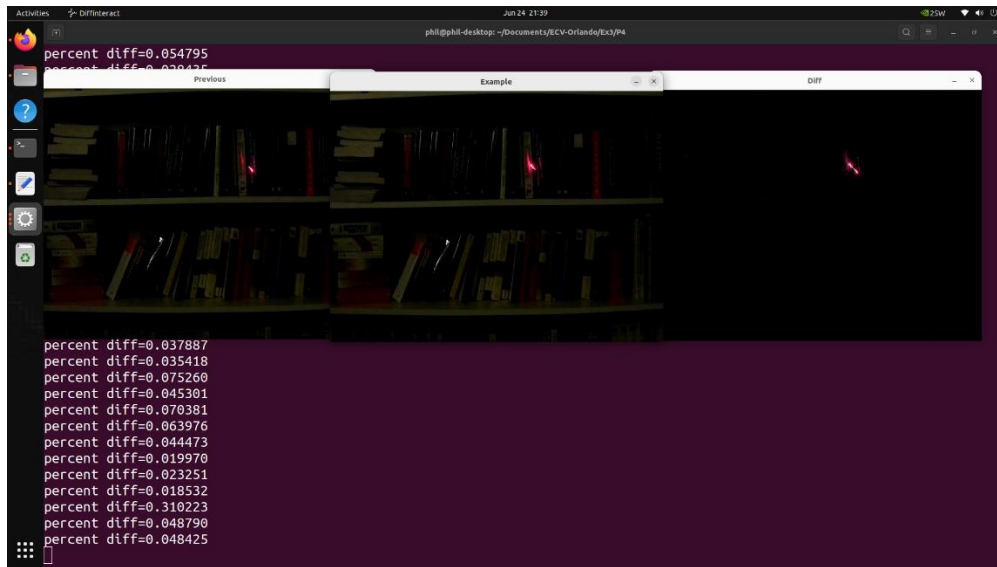


Figure (3) Background removal with frame difference.

5. Taking only red good and then re-encoding is good and has simple grayscale of the laser spot which makes the processing of the image with only one-color channel much faster and ability to do higher frame rate video in real time. To break the video into the red channel and then frame by frame I used OpenCV to split each frame into its channels and then save only the red channel to each frame pgm file. To try both I re-encoded the video to MPEG4 using the following ffmpeg command “ffmpeg -framerate 30 -i laserframe%04.pgm encodelaser.mpeg”. This resulted in 30 frames per second in the MPEG4 file and in gray scale since the single channel was displayed in grayscale.

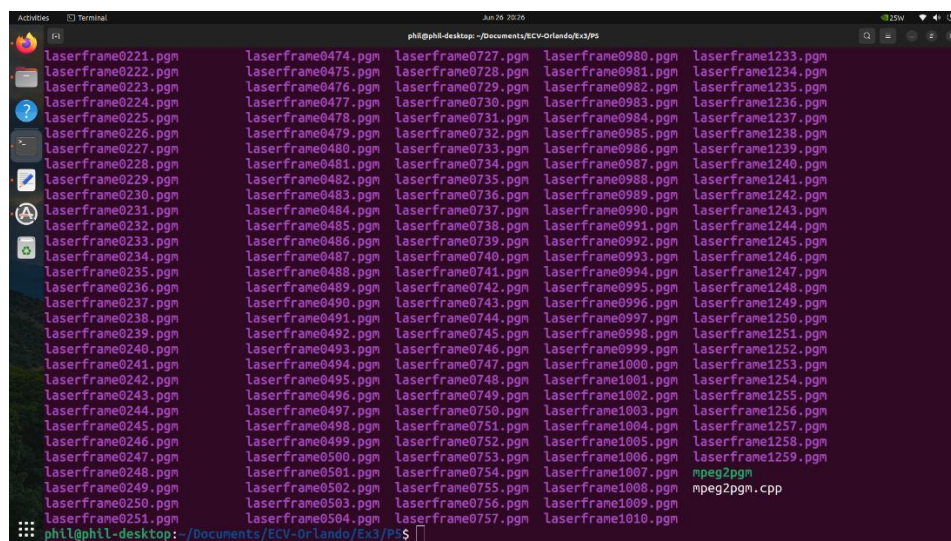


Figure (4) Terminal after video to frame-by-frame pgm conversion.

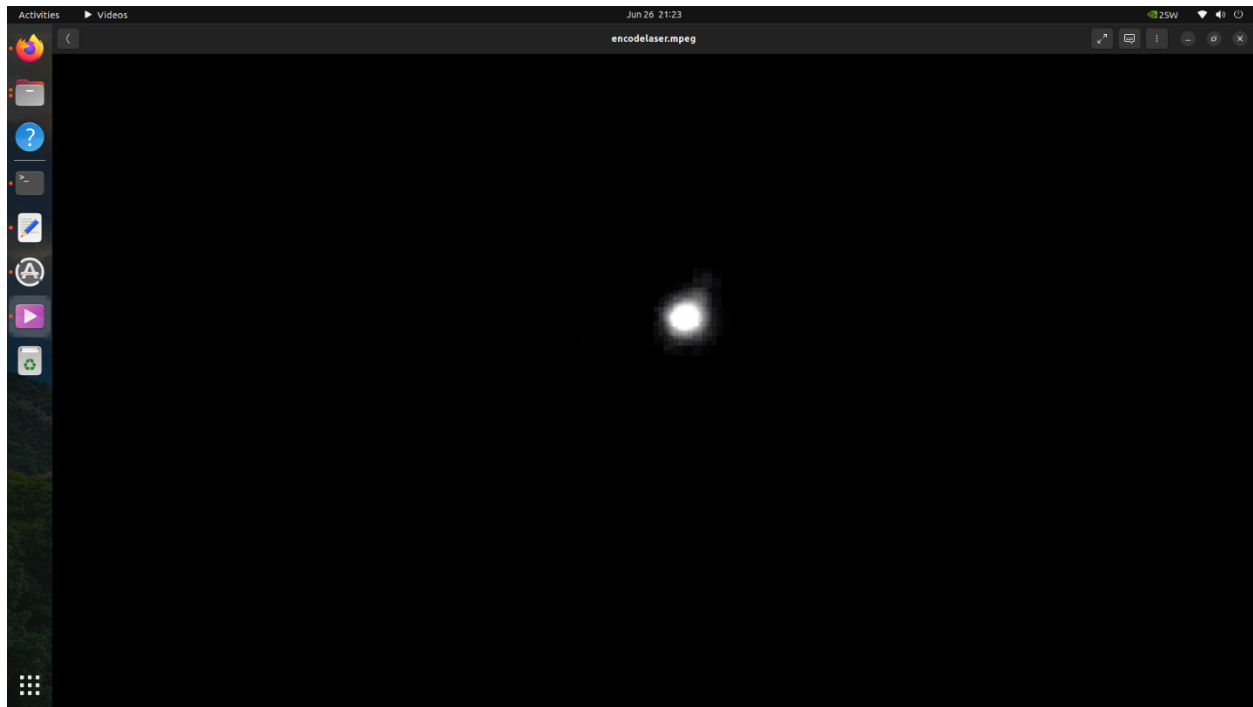


Figure (5) Screenshot of video re-encoded.

6. With the threshold and minimizing to one band of color, I was able to track the laser dot extremely well and even recognize when it was off the screen. I worked with the same OpenCV frame to take the video file of encoded grayscale dot and save individual frames. I then used an identical ffmpeg command as above to create the video again linked in my Github.